

Generating a Dasymetric Population Grid of the Twin Cities Metro Using PostGIS

Introduction

This project seeks to create a population grid of the Minneapolis - St Paul metropolitan area. The ultimate goal is to have a high quality hexagonal grid that can be employed for future analyses, especially in the realm of transportation planning. To that end, dasymetric filtering techniques will first be used to refine the input census geometries and then the refined area will be aggregated into a hexagonal grid. Hexagons have a few advantages as a geometry which makes them a better choice for transit analysis. The distance from the center to an edge is more consistent than with other tessellating shapes like squares or triangles, and they have a consistent area which makes standardizing statistical data easier. This hex grid setup was written in SQL to better facilitate efficient join operations with large datasets, and a repeatable process that can be maintained as new census and land use datasets are released.

Methods

A variety of data vintages will be integrated in order to leverage the relative strengths of each dataset, including:

- Population and race at the block level from the 2020 census counts.
<https://www.census.gov/data/developers/data-sets/decennial-census.html>
- Job counts at the block level from the 2023 LODES WAC job survey.
<https://lehd.ces.census.gov/data/>
- Education counts, car ownership, commute share, and total income at the block group level from the 2024 ACS 5 year survey.
<https://www.census.gov/data/developers/data-sets/acs-5year.html>
- Generalized land use from a 2020 Metropolitan Council dataset.
<https://gisdata.mn.gov/dataset/us-mn-state-metc-plan-general-land-use-2020>
- Building footprints from Bing Maps generated from imagery from 2019-2020.
https://tiles.arcgis.com/tiles/P3ePLMYs2RVChkXj/arcgis/rest/services/Microsoft_Building_Footprints/VectorTileServer

The overall order of operations is something like the following:

1. Load into PostGIS the 7 county census data, census geometry, LODES Jobs, building footprints, land use classifications, and an H3 uber hex grid generated in ArcGIS Pro.
2. Apportion block group level data to blocks based on the relative population of each block. This involved a simple join based on a string slice of the geoid column thanks to the nested nature of census geometries.
3. Prepare the datasets for computationally expensive spatial queries by subdividing the land use polygons, generating centroids for the building footprints, and saving spatial indexes. It was

essential to use ST_Subdivide() in order to make the following query computationally reasonable. The cost was verified with EXPLAIN calls so as to prevent getting stuck waiting for long inefficient spatial join queries.

4. Intersect the land use type with centroids of the building footprints and identify each with its census block GEOID. This sets the stage for apportionment later, as it enables step 6 to be refined and completed mostly without needing to consider geometry.

5. Build a table to categorize footprints based on land use; if the building is likely to be a job site, residence, or both. Storing it as a separate dictionary/lookup table is normalized better, because the category always directly depends on the land use. By storing it in a table, it could be modified as the apportionment logic was refined.

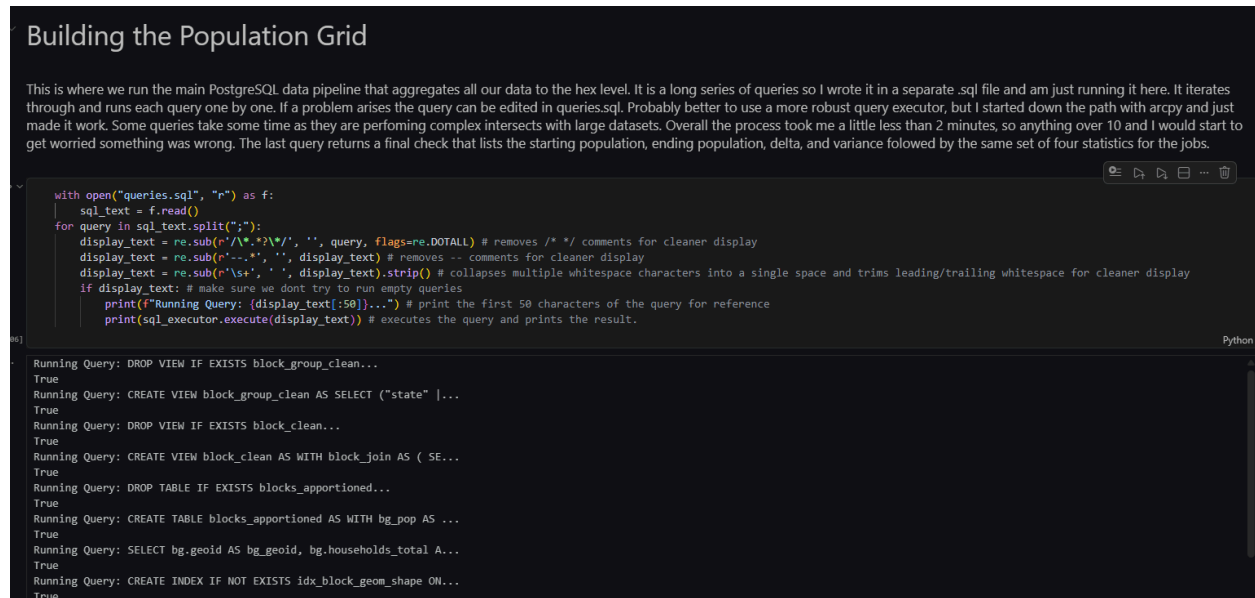
6. Apportion block level data to buildings based on classification category and relative area weight. Jobs go to commercial buildings and population characteristics go to residential buildings. Assigned mixed use buildings 20% of their area as commercial and 80% of their area as residential to reflect a common 4 floors of residential over 1 floor of retail building type, this is an assumption that could be refined. Employed several tiers of fallback to prevent population from disappearing in this step. Each successive fallback became less restrictive of what building types to allocate statistics, and eventually as a final fallback census blocks that have data but no buildings are included in the final building dataset to make sure no jobs or population are lost through the apportionment. This is less spatially accurate than the building footprints, but this was only employed in areas where either the land use data or building footprint data was likely not accurate enough to support the dasymetric filtering. The census data was treated as more trustworthy and if it indicated population should be present, that trumped a lack of proper buildings or zoning to support it.

7. Aggregate counts up from the buildings to a hex grid. Calculate ratio data from the totals, like percent of workers who drove alone or per capita income. Most of the tricky apportionment logic was applied in the previous step and this was more or less a straightforward sum of statistics of all buildings which had a center point within each hex.

8. Finally, verify population data was not lost in the process. A fractional degree of loss is expected due to floating point math and rounding, but significant loss should be investigated. If successful, a grid is ready for further analysis.

Figure 1

This is a snip from the jupyter notebook, of how queries were called:



```
Building the Population Grid

This is where we run the main PostgreSQL data pipeline that aggregates all our data to the hex level. It is a long series of queries so I wrote it in a separate .sql file and am just running it here. It iterates through and runs each query one by one. If a problem arises the query can be edited in queries.sql. Probably better to use a more robust query executor, but I started down the path with arcpy and just made it work. Some queries take some time as they are performing complex intersects with large datasets. Overall the process took me a little less than 2 minutes, so anything over 10 and I would start to get worried something was wrong. The last query returns a final check that lists the starting population, ending population, delta, and variance followed by the same set of four statistics for the jobs.

with open("queries.sql", "r") as f:
    sql_text = f.read()
    for query in sql_text.split(";"):
        display_text = re.sub(r"/\*/\*/", "", query, flags=re.DOTALL) # removes /* */ comments for cleaner display
        display_text = re.sub(r"--", "", display_text) # removes -- comments for cleaner display
        display_text = re.sub(r"\s+", " ", display_text).strip() # collapses multiple whitespace characters into a single space and trims leading/trailing whitespace for cleaner display
        if display_text: # make sure we dont try to run empty queries
            print(f"Running Query: {display_text[:50]}...") # print the first 50 characters of the query for reference
            print(sql_executor.execute(display_text)) # executes the query and prints the result.
```

Running Query: DROP VIEW IF EXISTS block_group_clean...
True
Running Query: CREATE VIEW block_group_clean AS SELECT ("state" |...
True
Running Query: DROP VIEW IF EXISTS block_clean...
True
Running Query: CREATE VIEW block_clean AS WITH block_join AS (SE...
True
Running Query: DROP TABLE IF EXISTS blocks_apportioned...
True
Running Query: CREATE TABLE blocks_apportioned AS WITH bg_pop AS ...
True
Running Query: SELECT bg.geoid AS bg_geoid, bg.households_total A...
True
Running Query: CREATE INDEX IF NOT EXISTS idx_block_geom_shape ON...
True

Data was initially collected using a jupyter notebook in order to call census table apis and gather hosted feature layers that contained the geometry. Once the data was staged in python it was loaded into a local install of PostgreSQL. Once all the data was assembled, the data pipeline was deployed by calling each query in turn programmatically in the jupyter notebook.

Challenges

There were several challenges involved in creating this data pipeline. The first was simply the choice of using jupyter notebooks and a cloned arcpy environment. This choice was dictated by a desire to leverage ArcGIS visualization techniques and the R5Py routing engine in future stages of the project, but it would have been much simpler to leverage other python to sql database libraries. There were several instances where it was a struggle to perform simple operations like loading a dataset into PostGIS and reading data back out in a usable format. Because the SQL query chain would remain the same in a different environment, this project will likely be transitioned to use a different library and python environment with more robust SQL support in the future.

Once the environment challenges were overcome the next challenge was optimizing the performance of complex spatial data operations. A query ran for over an hour before it needed to be interrupted early in pgAdmin when attempting to join land use with the building footprints. In this case, the biggest culprit was the land use polygons which categorized the entire metro into massive multipolygons, each with the same extent. This meant the query planner was unable to compare bounding boxes as a shortcut when joining to the buildings, and instead had to check every single building against the vertex dense geometry. Once ST_Subdivide() was used on the land use table, the query was able to process the three table spatial join to create

the buildings_enriched table within 30 seconds. After that experience, it became clear that any spatial query with datasets this large should probably first be run with an EXPLAIN to check if the spatial indexes are being properly used and the compute cost is reasonable. I also used ANALYZE commands at key times throughout the pipeline to ensure that the query planner was aware of the indexes that were available to speed up operations.

The final challenge was designing the set of queries, such that no population or jobs were lost throughout the multiple layers of apportionment. In order to overcome this, a series of fallback options was implemented when apportioning data to the building footprints for cases where there were gaps in the building footprint or the land use data was misclassifying populated areas as undeveloped. The mismatch of the data vintages made errors in the spatial distribution inevitable, so preventing data loss was prioritized. To this end, in cases where there were no valid buildings to apportion data to, the original block geometry was used as a final fallback.

Results

In the end, a population grid was successfully created, and the pipeline was optimized such that the entire conversion from raw data to apportioned hex grid takes less than a few minutes. The grid contains multiple census statistics that will be useful for future analysis tasks, such as a transit accessibility analysis. Fallback logic was successful in preventing significant data leakage, and the variance was functionally zero for both total population and total jobs.

Verification Table

This table was the result of the final sanity check query and shows that there was very little change in total population or job count.

Final population:	3163104.0000000177
Starting population:	3163104
Delta:	1.7695128917694092e-08
Variance:	0.0

Final jobs:	1794824.9999999912
Starting jobs:	1794825
Delta:	-8.847564458847046e-09
Variance:	0.0

Overall, this process highlights the relative benefit of table manipulations using SQL over running the same operations using dissolve, spatial join, join, and field calculator functions in

ArcGIS Pro. In a SQL database the user is able to execute multiple operational steps in one query, and that query can be optimized to provide a streamlined data pipeline capable of leveraging the strengths of the query planner to avoid costly operations when possible. The case when logic of fallback options when the ideal apportionment was not available was much more intuitive than trying to cross check the tables after a spatial join and appending missing blocks into the dataset as needed. Well written SQL is relatively easy to follow logically, and a whole process can be documented query by query. In a python environment memory can be a limiting factor when working with large datasets, but the postgres database handled the massive metro building footprint and land use type datasets with ease.

Population Map

This is a picture of the population grid that was generated with this process:

